Операционные системы

Лекция 8

Синхронизация

План лекции

- Необходимость синхронизации и гонки, понятие «критическая секция»
- Задачи синхронизации
- Средства синхронизации потоков одного процесса: на основе использования системы прерываний, блокирующие переменные и семафоры
- Синхронизирующие объекты ОС для синхронизации потоков разных процессов

Синхронизация

Как мы уже выяснили, довольно часто процессам необходимо взаимодействовать с другими процессами и для этого ОС предоставляет некоторые механизмы взаимодействия IPC

Однако тут могут возникнуть как минимум 2 вопроса:

- Не создаст ли такое взаимодействие помех в работе каждого из них?
- Есть ли какая-то строгая последовательность которой стоит придерживаться в выполнении таких процессов?

Синхронизация

Потребность в синхронизации потоков возникает только в мультипрограммной операционной системе и связана с совместным использованием несколькими процессами аппаратных и информационных ресурсов вычислительной системы

Выполнение потока в мультипрограммной среде всегда имеет ***асинхронный характер***. Очень сложно с полной определенностью сказать, на каком этапе выполнения будет находиться процесс в определенный момент времени

Потоки в общем случае (когда программист не предпринял специальных мер по их синхронизации) протекают независимо, асинхронно друг другу. Это справедливо как по отношению к потокам одного процесса, выполняющим общий программный код, так и по отношению к потокам разных процессов, каждый из которых выполняет собственную программу

Синхронизация

Любое взаимодействие потоков, связанное с разделением ресурсов или обменом данными, требует их **синхронизации**

Синхронизация заключается в согласовании скоростей потоков путем приостановки одного или нескольких из них до наступления некоторого события и последующей активизации при наступлении этого события

Синхронизация является важной задачей как в разрезе некоторых программных ресурсов, так и в случае когда речь идёт об аппаратных ресурсах

Синхронизация

Для синхронизации потоков прикладных программ программист может использовать как **собственные средства и приемы синхронизации**, так и **средства операционной системы**

Например, два потока одного прикладного процесса могут координировать свою работу с помощью доступной для них обоих глобальной, логической переменной, которая устанавливается в единицу при осуществлении некоторого события, например выработки одним потоком данных, нужных для продолжения работы другого

Однако во многих случаях более эффективными или даже единственно возможными являются средства синхронизации, предоставляемые операционной системой в форме системных вызовов

Задача синхронизации

Пренебрежение вопросами синхронизации в многопоточной системе может привести к неправильному решению задачи или даже к краху системы. Рассмотрим, например, задачу ведения базы данных клиентов некоторого предприятия:

Каждому клиенту отводится отдельная запись в базе данных, в которой среди прочих полей имеются поля Заказ и Оплата

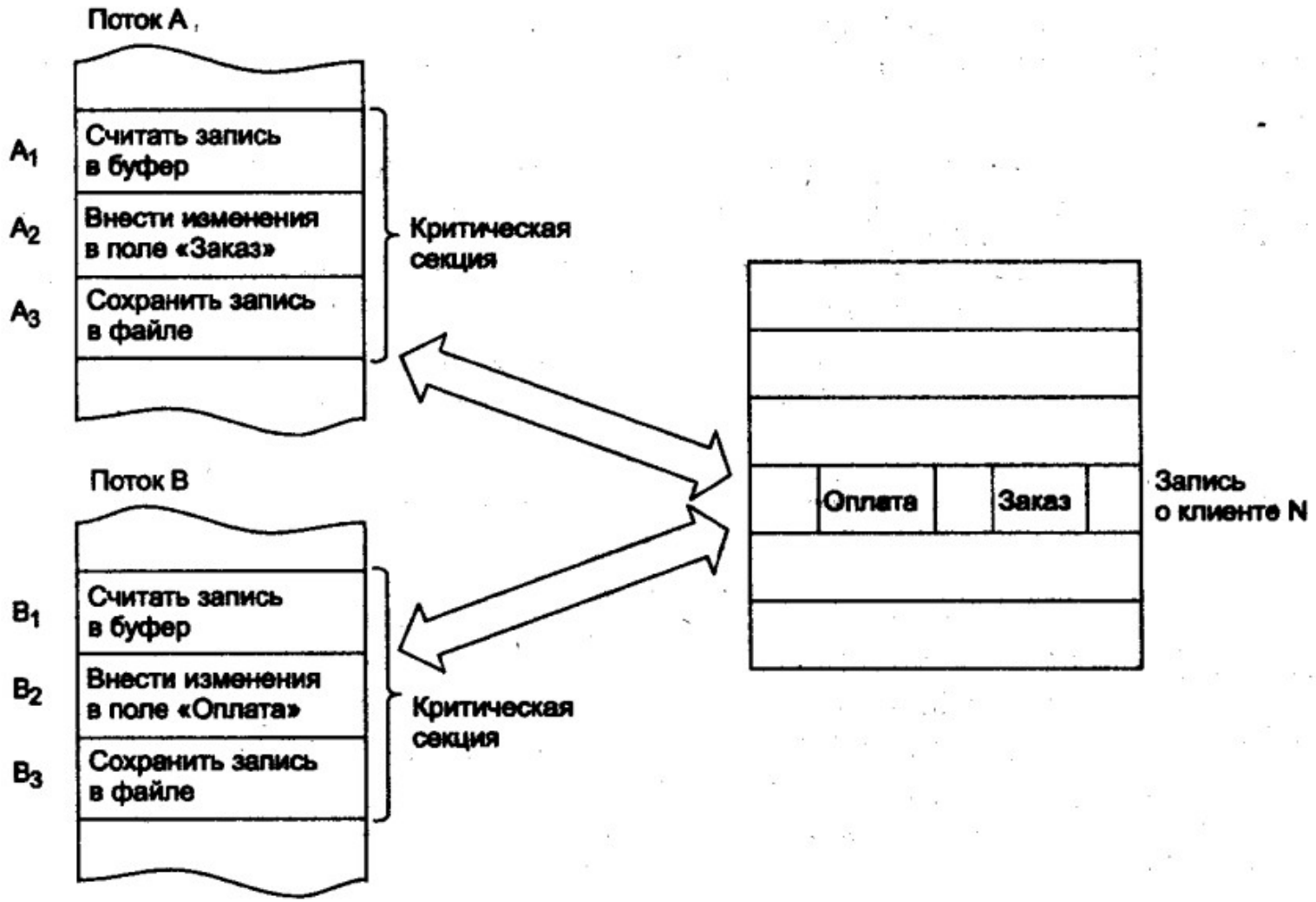
Программа, ведущая базу данных, оформлена как единый процесс, имеющий несколько потоков, в том числе поток А, который заносит в базу данных информацию о заказах, поступивших от клиентов, и поток В, который фиксирует в базе данных сведения об оплате клиентами выставленных счетов

Задача синхронизации

Оба эти потока совместно работают над общим файлом базы данных, используя однотипные алгоритмы, включающие три шага:

1. Считать из файла базы данных в буфер запись о клиенте с заданным идентификатором
2. Внести новое значение в поле Заказ (для потока A) или Оплата (для потока B)
3. Вернуть модифицированную запись в файл базы данных

Задача синхронизации



Задача синхронизации

Предположим, что в некоторый момент поток A обновляет поле Заказ записи о клиенте N:

1. Для этого он считывает эту запись в свой буфер (шаг A1)

2. Модифицирует значение поля Заказ (шаг A2), но записать запись в базу данных (шаг A3) не успевает, так как его выполнение прерывается, например, вследствие завершения кванта времени

Предположим также, что потоку B также потребовалось внести, сведения об оплате относительно того же клиента N. Когда подходит очередь потока B, он:

3. Успевает считать запись в свой буфер (шаг B1) и выполнить обновление поля

4. Оплата (шаг B2), а затем прерывается

Заметим, что в буфере у потока B находится запись о клиенте N, в которой поле Заказ имеет прежнее, не измененное значение

Задача синхронизации

Когда в очередной раз управление будет передано потоку A, то он:

5. Продолжая свою работу, сделает запись о клиенте N с модифицированным полем Заказ в базу данных (шаг A3)

После прерывания потока A и активизации потока B последний:

6. Запишет в базу данных поверх только что обновленной записи о клиенте N свой вариант записи, в которой обновлено значение поля Оплата

Таким образом, в базе данных будут зафиксированы сведения о том, что клиент N произвел оплату, но информация о его заказе окажется потерянной (**ситуация *a***)

Задача синхронизации

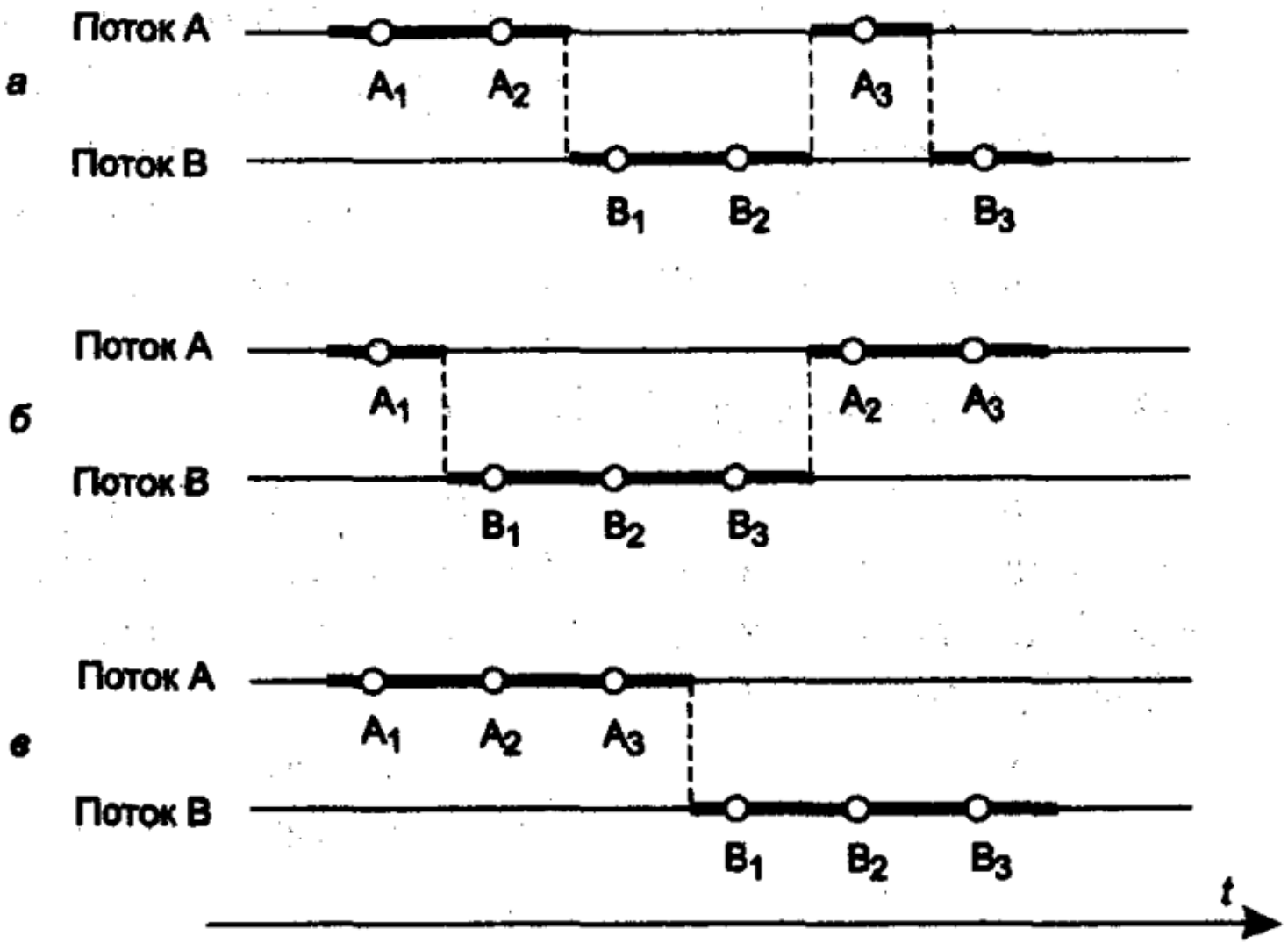
Сложность проблемы синхронизации кроется в нерегулярности возникающих ситуаций

Так, в предыдущем примере можно представить и другое развитие событий: могла быть потеряна информация не о заказе, а об оплате (**ситуация *б***). Или, напротив, все исправления могли быть успешно внесены (**ситуация *в***)

Все определяется взаимными скоростями потоков и моментами их прерывания. Поэтому отладка взаимодействующих потоков является сложной задачей

Результаты большинства прогонов могут быть вполне приемлемыми, но до поры до времени, пока не наступит тот самый редкий случай, когда произойдет нечто таинственное и необъяснимое. К сожалению, с ростом параллелизма из-за все большего количества ядер состязательные ситуации встречаются все чаще

Задача синхронизации



Состояние гонки

Ситуации подобные той, когда два или более потоков обрабатывают разделяемые данные, а конечный результат зависит от соотношения скоростей потоков, называются **гонками**

Состояние гонки – ситуация, когда несколько потоков или процессов читают и записывают элемент общих данных и конечный результат зависит от относительного времени выполнения этих потоков

Ситуация, когда два или более потока считывают или записывают какие-нибудь общие данные, а окончательный результат зависит от того, какой поток и когда именно выполняется, называется **состязательной ситуацией**

Состояние гонки

Хотя как мы видим в разных источниках подобные ситуации называются по разному, суть от этого не меняется

Как же избежать состязательной ситуации?

Ключом к предупреждению проблемы в этой и во многих других ситуациях использования общей памяти, общих файлов и вообще чего-нибудь общего может послужить определение способа, при котором в каждый конкретный момент времени доступ к общим данным для чтения и записи может получить только один поток

Иными словами, нам нужен способ **взаимного исключения**, то есть некий способ, обеспечивающий правило, при котором если общие данные или файл используются одним потоком, возможность их использования всеми другими потоками исключается

Критическая секция

Та часть программы, в которой используется доступ к общей памяти, называется **критической областью** или **критической секцией**

Критическая секция – это часть программы, результат выполнения которой может непредсказуемо меняться, если переменные, относящиеся к этой части программы, изменяются другими потоками в то время, когда выполнение этой части еще не завершено

Критические данные

Критическая секция всегда определяется по отношению к определенным **критическим данным**, при несогласованном изменении которых могут возникнуть нежелательные эффекты

Во всех потоках, работающих с критическими данными, должна быть определена критическая секция

Заметим, что в разных потоках критическая секция состоит, в общем случае, из разных последовательностей команд

Критическая секция

Если бы удалось всё выстроить таким образом, чтобы никакие два потока не находились одновременно в своих критических областях, это позволило бы избежать гонки

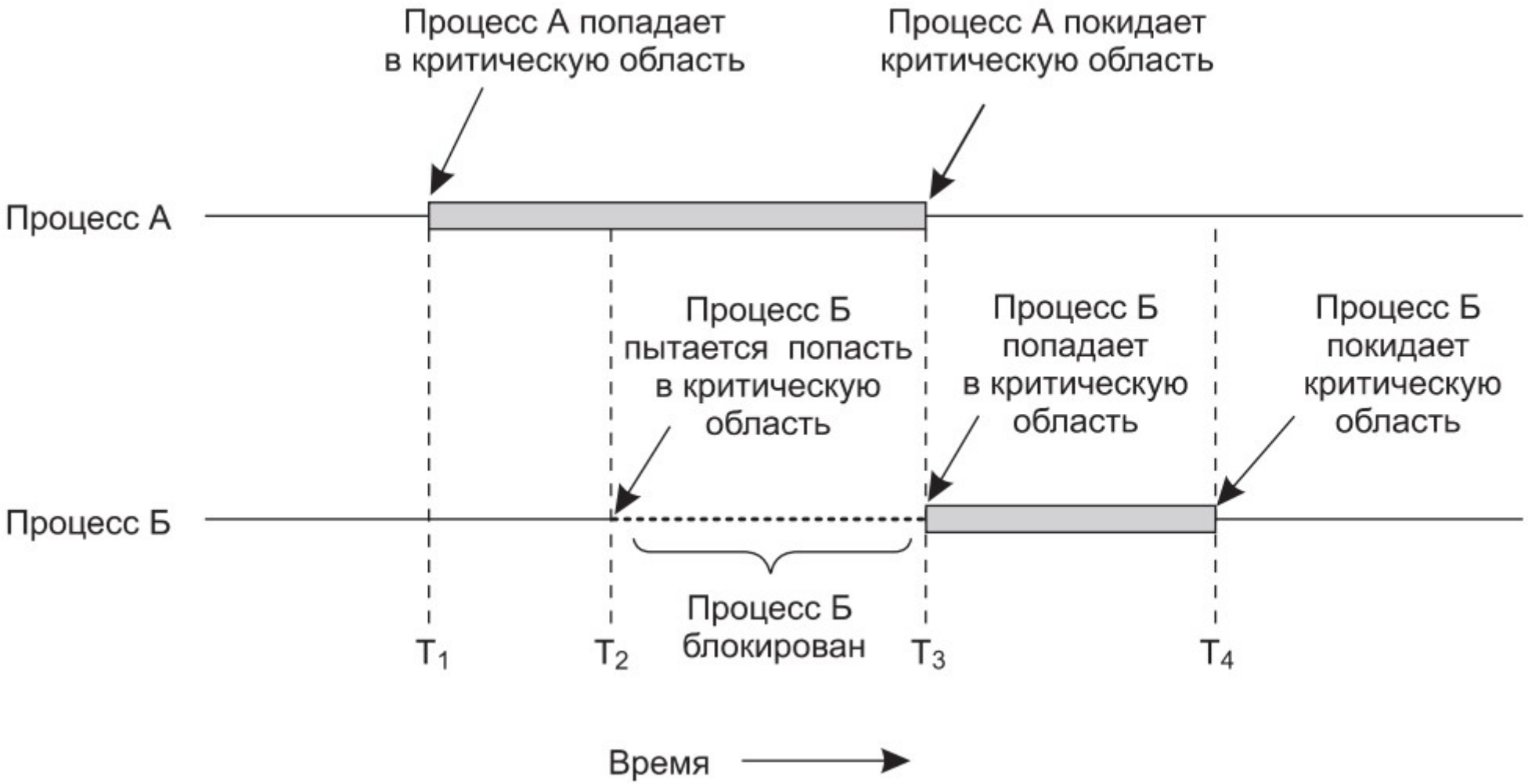
Хотя выполнение этого требования позволяет избежать состязательных ситуаций, его недостаточно для того, чтобы параллельные потоки правильно выстраивали совместную работу и эффективно использовали общие данные

Критическая секция

Для приемлемого решения необходимо соблюдение четырех условий:

1. Два потока не могут одновременно находиться в своих критических областях
2. Не должны выстраиваться никакие предположения по поводу скорости или количества центральных процессоров
3. Никакие потоки, выполняемые за пределами своих критических областей, не могут блокироваться любым другим потоком
4. Потоки не должны находиться в вечном ожидании входа в свои критические области

Критическая секция



Синхронизация

Перед тем как переходить к возможным вариантам механизмов синхронизации, рассмотрим основные задачи которые отражают те проблемы, которые нам возможно предстоит решить в мультизадачной среде

Типовыми задачами в области синхронизации являются:

- Задача взаимного исключения
- Задача «производитель-потребитель»
- Задача «читатели-писатели»
- Задача «клиент-официант»
- Задача «обедающие философы»

Задача взаимоисключений

Первую задачу мы с вами по сути рассматривали всё это время!

Задача взаимного исключения состоит в следующем:

Задача взаимного исключения заключается в необходимости гарантировать, что в каждый момент времени только один поток может выполнять критическую секцию кода, работающую с общим (разделяемым) ресурсом

Когда несколько параллельных потоков пытаются одновременно:

- читать/записывать данные в общую переменную
- изменять структуру данных (список, дерево, хеш-таблицу)
- обращаться к аппаратному ресурсу (порт, устройство)
- модифицировать файл или запись в базе данных

возникает **неопределённость результата** из-за непредсказуемого порядка выполнения инструкций

Задача взаимоисключений

Успешное решение данной проблемы должно обладать по крайней мере двумя следующими свойствами:

- Оно должно обеспечивать **взаимное исключение**: в **критическую секцию** в каждый момент времени может находиться только один поток
- Оно должно быть свободно от **тупиковых ситуаций (deadlocks)**: если потоки пытаются войти в критическую секцию, то хотя бы один из них в конечном итоге должен успешно это сделать – при условии, что ни один поток не остаётся в критическом участке постоянно

Задача взаимоисключений

Осуществление взаимных исключений создает две дополнительные проблемы:

Первая проблема – **взаимная блокировка** (deadlock)

Вторая проблема – **голодание** (starvation)

Взаимная блокировка – ситуация, когда два и более потоков не в состоянии работать, поскольку каждый из потоков ожидает выполнения некоторого действия другим процессом

Голодание – ситуация, когда запуск потока пропускается планировщиком бесконечное количество раз; хотя поток готов работать, он никогда не выбирается

Задача взаимоисключений

Рассмотрим, например, два потока (P1 и P2) и два ресурса (R1 и R2)

Предположим, что каждому потоку для выполнения части своих функций требуется доступ к обоим ресурсам. Тогда возможно возникновение следующей ситуации: операционная система выделяет ресурс R1 потоку P2, а ресурс R2 – потоку P1

В результате каждый поток ожидает получения одного из двух ресурсов; при этом ни один из них не освобождает уже имеющийся у него ресурс, ожидая получения второго ресурса для выполнения функций, требующих наличия двух ресурсов

В результате процессы оказываются взаимно заблокированными

Задача взаимоисключений

Предположим, что у нас имеются три потока (P1, P2, P3), каждому из которых периодически требуется доступ к ресурсу R

Представим ситуацию, в которой P1 обладает ресурсом, а P2 и P3 приостановлены в ожидании освобождения ресурса. После выхода P1 из критического раздела доступ к ресурсу будет получен одним из потоков P2 и P3

Пусть операционная система предоставила доступ к ресурсу R потоку P3. Пока он работает с ресурсом, доступ к ресурсу вновь требуется потоку P1. В результате после освобождения ресурса потоком P3 может оказаться, что операционная система вновь предоставила доступ к ресурсу потоку P1; тем временем потоку P3 вновь требуется доступ к ресурсу R

Таким образом, теоретически возможна ситуация, в которой поток P2 никогда не получит доступа к требуемому ему ресурсу, несмотря на то что никакой взаимной блокировки в этом случае нет

Задача взаимоисключений

Как уже говорилось поддержка синхронизации может осуществляться как на программном, так и на аппаратном уровнях

Среди аппаратных подходов к решению вопроса взаимоисключений можно отнести:

- Отключение прерываний
- Специальные машинные команды

Среди программных подходов стоит отметить:

- Мьютексы
- Атомарные операции
- Спинлоки

Отключение прерываний

В однопроцессорных системах простейшим решением является запрещение всех прерываний каждым потоком сразу после входа в критическую область и их разрешение сразу же после выхода из критической области

При запрещении прерываний не могут осуществляться никакие прерывания по таймеру. Поскольку центральный процессор переключается с одного потока на другой в результате таймерных или каких-нибудь других прерываний, то при выключенных прерываниях он не сможет переключиться на другой поток

Поскольку процесс запретил прерывания, он может исследовать и обновлять общую память, не опасаясь вмешательства со стороны любого другого потока

Отключение прерываний

Поскольку критический раздел не может быть прерван, выполнение взаимоисключения гарантируется. Однако цена такого подхода высока

Этот подход не слишком привлекателен, поскольку абсолютно неразумно давать пользовательским потокам полномочия выключать все прерывания. Представьте, что получится, если один из них выключил и не включил прерывания?

Это может вызвать крах всей системы. Более того, если мы имеем дело с многопроцессорной системой (с двумя или, может быть, несколькими центральными процессорами), запрещение прерываний действует только на тот центральный процессор, на котором выполняется запретительная инструкция. Все остальные процессоры продолжат свою работу и смогут обращаться к общей памяти

Отключение прерываний

В то же время запрещение прерываний всего на несколько инструкций зачастую является очень удобным средством для самого ядра, когда оно обновляет переменные или списки

К примеру, когда прерывание происходит в момент изменения состояния списка готовых процессов, может сложиться состязательная ситуация

Вывод здесь следующий: запрещение прерываний в большинстве своем является полезной технологией внутри самой операционной системы, но не подходит в качестве универсального механизма взаимных блокировок для пользовательских потоков (не путать с потоками уровня пользователя)

Отключение прерываний

В многопроцессорной конфигурации несколько процессоров разделяют доступ к общей основной памяти

В этом случае отсутствует отношение «ведущий/ведомый» – процессоры работают независимо, «на равных», и не существует механизма прерывания, на котором могли бы основываться взаимоисключения

На уровне аппаратного обеспечения, обращение к ячейке памяти исключает любые другие обращения к той же ячейке

Основываясь на этом принципе, разработчики процессоров предлагают ряд машинных команд, которые за один цикл выборки команды **атомарно** выполняют над ячейкой памяти два действия, такие как чтение и запись или чтение и проверка значения

Атомарная операция

Атомарная операция – функция или действие, реализованное как последовательность из одной или нескольких инструкций, представляющихся неделимыми; т.е. никакой другой процесс не может увидеть промежуточное состояние или прервать выполняющуюся операцию

Гарантируется выполнение последовательности инструкций как единой группы (либо не выполнение ни одной из них, без влияния на видимое состояние системы)

Атомарность гарантирует изоляцию от параллельно выполняющихся потоков

Машинные инструкции

Существует две наиболее часто реализуемые машинные инструкции:

- **Команда проверки и присваивания**

Эта атомарная команда состоит из двух частей – **сравнение значений** в ячейке памяти и тестового значения, и, если эти значения совпадают, выполняется **присваивание**. Вся функция выполняется атомарно, т.е. она не может быть прервана (например, команда BTS* в x86 процессорах)

- **Команда обмена**

Эта команда обменивает содержимое регистра и ячейки памяти (например, команда XCHG в x86 процессорах)

* Без модификатора LOCK не применима в реализации взаимоисключений

Машинные инструкции

```
int compare_and_swap(int* word, int testval, int newval)
{
    int oldval;
    oldval = *word;
    if (oldval == testval) *word = newval;
    return oldval;
}
```

```
void exchange(int* register, int* memory)
{
    int temp;
    temp = *memory;
    *memory = *register;
    *register = temp;
}
```

Машинные инструкции

```
/* program mutualexclusion */
const int n = /* Количество процессов */;
int bolt;
void P(int i)
{
  while (true)
  {
    while(compare_and_swap(&bolt, 0, 1)
          == 1)
          /* Ничего не делать */;

    /* Критический участок */;
       bolt = 0;
    /* Остальной код */;
  }
}
void main()
{
    bolt = 0;
    parbegin(P(1), P(2), ... , P(n));
}
```

а) Команда сравнения и обмена

```
/* program mutualexclusion */
int const n = /* Количество процессов */;
int bolt;
void P(int i)
{
  while (true)
    int keyi = 1;
    do exchange(&keyi, &bolt)
    while (keyi != 0);
    /* Критический участок */;
    bolt = 0;
    /* Остальной код */;
  }
}

void main()
{
    bolt = 0;
    parbegin(P(1), P(2), ..., P(n));
}
```

б) Команда обмена

Машинные инструкции

Свойства подхода, основанного на использовании машинных инструкций:

- Применим к любому количеству потоков при наличии как одного, так и нескольких процессоров, совместно использующих основную память
- Очень прост, а потому легко проверяем
- Может использоваться для поддержки множества критических участков; каждый из них может быть определен при помощи собственной переменной

Машинные инструкции

Однако у такого подхода имеются и серьезные недостатки:

- Используется **пережидание занятости**. Следовательно, в то время как поток находится в ожидании доступа к критическому участку, он продолжает потреблять процессорное время
- Возможно **голодание**. Если процесс покидает критический участок, а входа в него ожидают несколько других процессов, то выбор ожидающего процесса произволен. Следовательно, может оказаться, что какой-то из процессов будет ожидать входа в критический участок бесконечно
- Возможна **взаимоблокировка**. Особенно в однопроцессорных системах с планированием на основе приоритетов. Например, поток с более низким приоритетом войдёт в секцию (выполнит compare_and_swap). Первый поток прерывается вторым потоком с высоким приоритетом и тот войдёт в состояние пережидания занятости, а первый поток уже не будет выбран для исполнения в виду своего низкого приоритета

Активное ожидание

Термин **пережидание занятости** (busy waiting, spin waiting) относится к методике, в соответствии с которой процесс до тех лор, пока не получит разрешение войти в критический участок, не может ничего делать, кроме как выполнять инструкции по проверке соответствующей переменной для получения разрешения на вход

Также данный подход носит название **активным ожиданием**

Данный принцип лежит в основе реализации **спин-блокировок (спинлоков)**

Данный подход активно используется в ситуациях когда известно что ожидание будет непродолжительным

Блокирующие переменные

Самым простым программным способом добиться взаимного исключения потоков одного процесса являются **глобальные блокирующие переменные**

Каждому набору критических данных ставится в соответствие двоичная переменная, которой поток присваивает значение 0, когда он входит в критическую секцию, и значение 1, когда он ее покидает

Блокирующие переменные могут использоваться не только при доступе к разделяемым данным, но и при доступе к разделяемым ресурсам любого вида

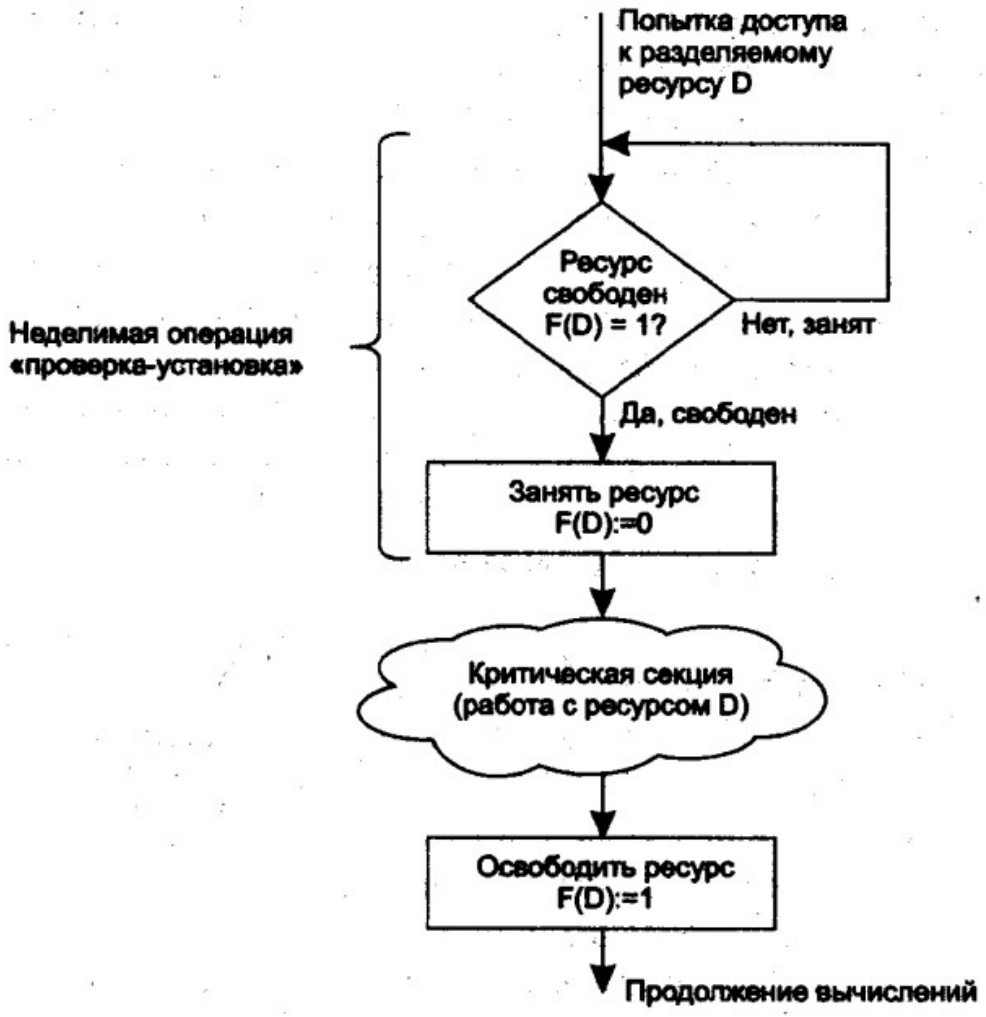
Блокирующие переменные

Алгоритм следующий:

1. Перед входом в критическую секцию поток проверяет, не работает ли уже какой-нибудь поток с данными D

2. Если переменная F(D) установлена в 0, то данные заняты, и проверка циклически повторяется. Если же данные свободны, то есть 1, тогда значение переменной F(D) устанавливается в 0, и поток входит в критическую секцию

3. После того как поток выполнит все действия с данными D, значение переменной F(D) снова устанавливается равным 1



Блокирующие переменные

Однако следует заметить, что одно ограничение на прерывания все же имеется. Нельзя прерывать поток между выполнением операций проверки и установки блокирующей переменной

Поясним это. Пусть в результате проверки переменной поток определил, что ресурс свободен, но сразу после этого, не успев установить переменную в 0, был прерван. За время его приостановки другой поток занял ресурс, вошел в свою критическую секцию, но также был прерван, не завершив работы с разделяемым ресурсом

Когда управление было возвращено первому потоку, он, считая ресурс свободным, установил признак занятости и начал выполнять свою критическую секцию. Таким образом, был нарушен принцип взаимного исключения, что потенциально может привести к нежелательным последствиям

Блокирующие переменные

Во избежание подобных проблем в системе команд многих компьютеров предусмотрена единая, неделимая команда анализа и присвоения значения логической переменной (BTS, BTR и BTC)

Если же таких команд нету, то можно использовать запрет прерываний на протяжении всей операции проверки и установки

Так как данный программный способ по сути является аналогичным описанному ранее аппаратному (сравнение и присваивание), то и проблемы остаются такими же

Чтобы потоки не занимались пережиданием занятости, в некоторых ОС вместо этого потоки переводятся в состояние ожидания

Критические секции

Одними из таких ОС является всё семейство Windows NT с их механизмом критических секций

Для работы с критической секцией в WinAPI существуют следующие функции:

- Инициализация секции – **InitializeCriticalSection**
- Занятие секции – **EnterCriticalSection**
- Попытка занять секцию – **TryEnterCriticalSection**
- Выход из секции – **LeaveCriticalSection**
- Очистка объекта секции – **DeleteCriticalSection**

В POSIX прямого аналога критическим секциям из Windows нету, но чем-то похожим является **futex**

Критические секции

```
#include <Windows.h>
#include <iostream>

DWORD pid = NULL;
DWORD WINAPI  ChildThread();

CRITICAL_SECTION cs;

int main()
{
    pid = GetCurrentProcessId();

    DWORD tid = GetCurrentThreadId();
    DWORD  ChildId = NULL;
    HANDLE hChild = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)ChildThread, NULL, 0, &ChildI

    InitializeCriticalSection(&cs);

    EnterCriticalSection(&cs);
    for (unsigned long long i = 0; i < 100; i++)
    {
        std::cout << "PID = " << pid << ", Parent Thread: " << "tid-" << tid << ": " << i << "\
        Sleep(40);
    }
    LeaveCriticalSection(&cs);

    WaitForSingleObject(hChild, INFINITE);

    DeleteCriticalSection(&cs);

    CloseHandle(hChild);
    return 0;
}
```

```
DWORD WINAPI  ChildThread()
{
    DWORD tid = GetCurrentThreadId();

    EnterCriticalSection(&cs);
    for (unsigned long long i = 0; i < 100; i++)
    {
        std::cout << "PID = " << pid << ", Child
        Sleep(40);
    }
    LeaveCriticalSection(&cs);

    return 0;
}
```

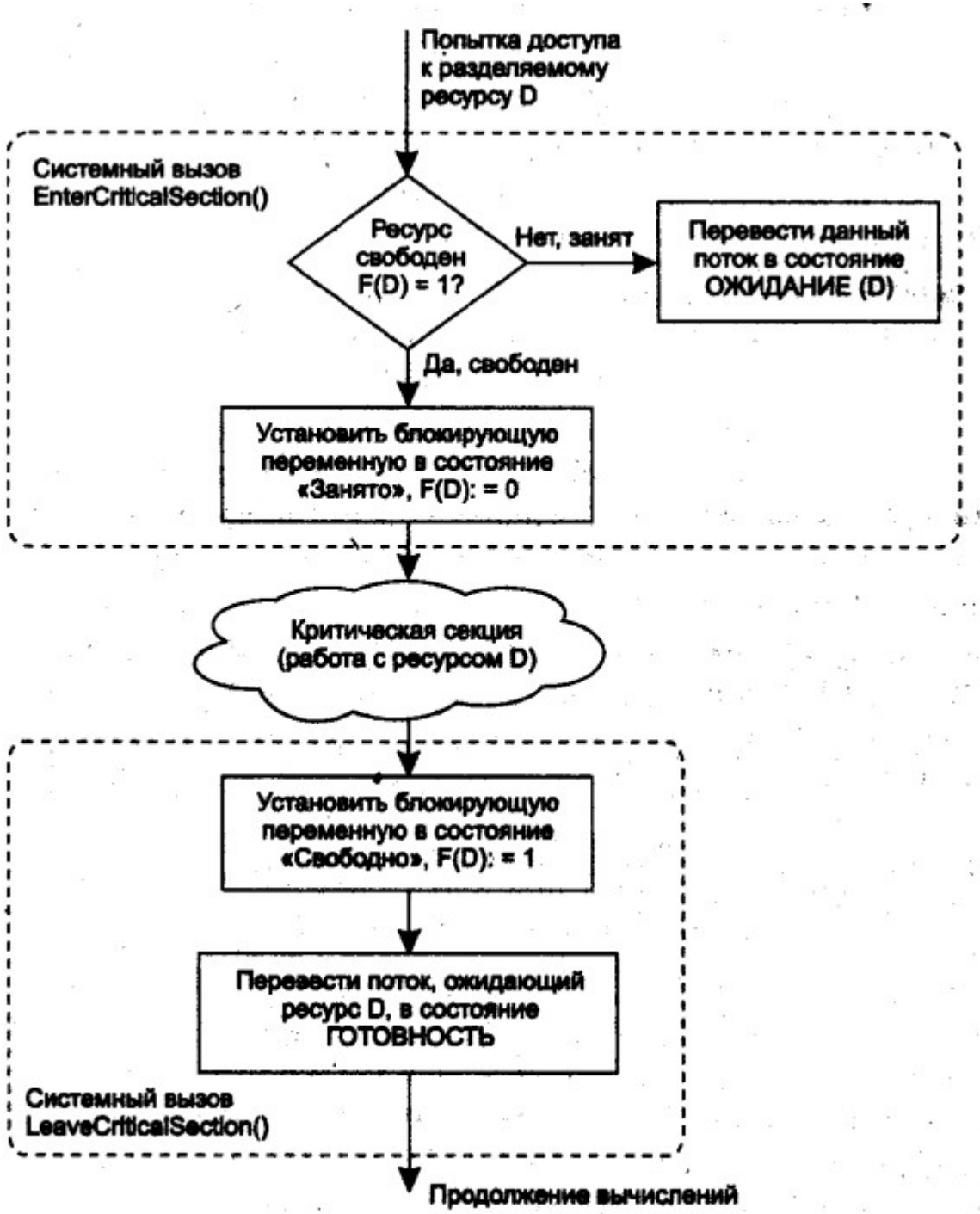
```
Консоль отладки Microsoft Visual Studio
PID = 860, Parent Thread: tid-16056: 0
PID = 860, Parent Thread: tid-16056: 1
PID = 860, Parent Thread: tid-16056: 2
PID = 860, Parent Thread: tid-16056: 98
PID = 860, Parent Thread: tid-16056: 99
PID = 860, Child  Thread: tid-14884: 0
PID = 860, Child  Thread: tid-14884: 1
PID = 860, Child  Thread: tid-14884: 2
```

Критические секции

Критическая секция – советы:

- На каждый разделяемый ресурс используйте отдельную структуру CRITICAL_SECTION
- Одновременный доступ к нескольким ресурсам

```
DWORD WINAPI ThreadFunc(PVOID pvParam) {
   EnterCriticalSection(&g_csNums);
   EnterCriticalSection(&g_csChars);
   // в этом цикле нужен одновременный доступ к обоим ресурсам
   for (int x = 0; x < 100; x++) g_nNums[x] = g_cChars[x];
   LeaveCriticalSection(&g_csChars);
   LeaveCriticalSection(&g_csNums);
   return(0);
}
```



Мьютексы

Другим примитивом подобным критическим секциям является объект «**Мьютекс**» (**mutex** – **mut**ual **ex**clusion)

По сути идентичен по своей сути объекту критической секции за исключением одного факта – он позволяет синхронизировать потоки **разных** процессов

Соответственно является объектом ядра ОС в виду необходимости работы с различными процессами

Соответственно для работы требует системных вызовов, а значит более дорогой в использовании по сравнению с секцией (касается только Windows)

Существует в рамках обеих рассматриваемых ОС: Windows и Linux

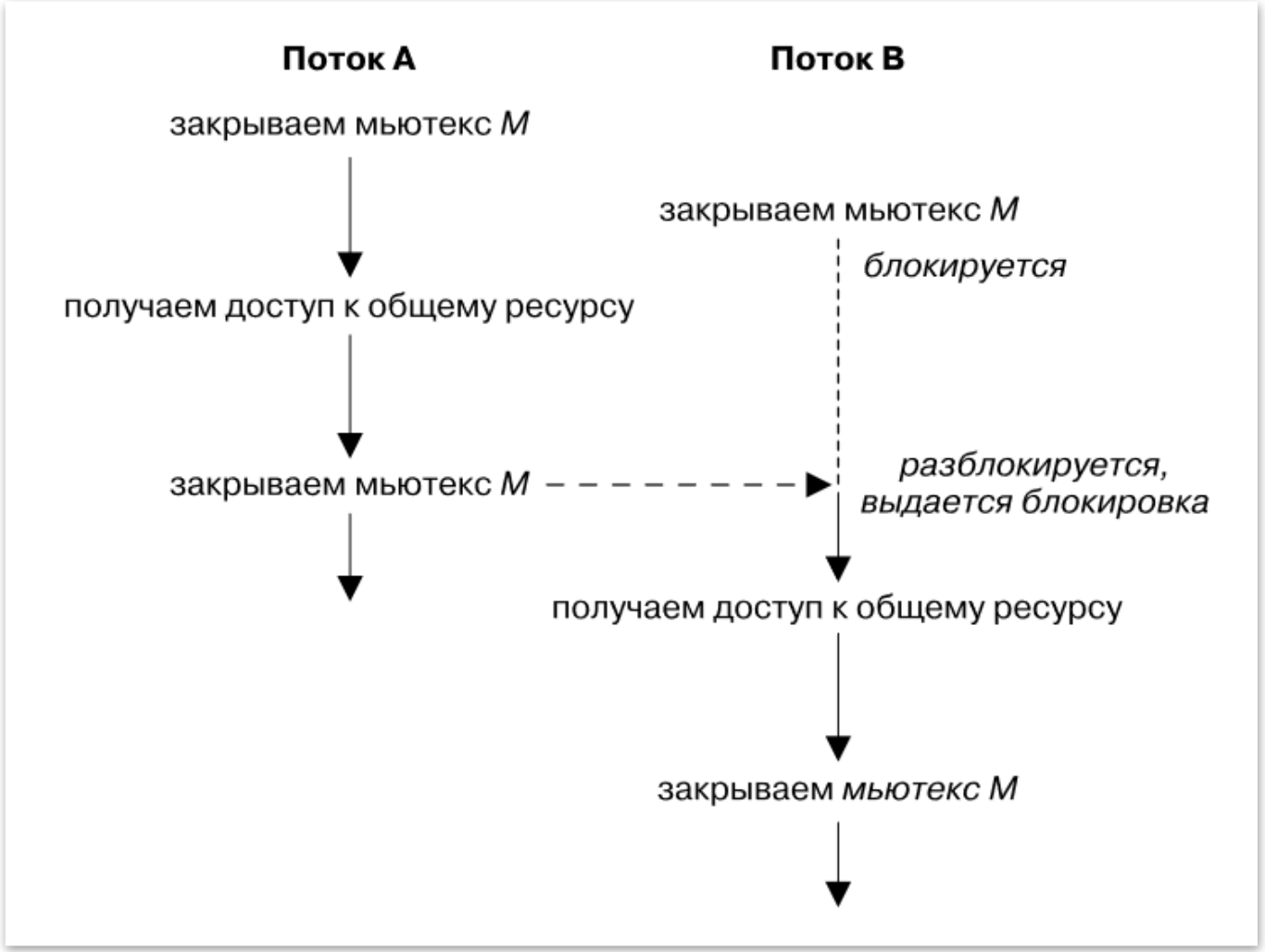
Мьютексы

Для мьютексов определены следующие правила:

- Если его идентификатор потока равен 0 (у самого потока не может быть такой идентификатор), мьютекс не захвачен ни одним из потоков и находится в свободном состоянии
- Если его идентификатор потока не равен 0, мьютекс захвачен одним из потоков и находится в занятом состоянии

Мьютексы

Использование мьютекса для защиты критического участка



Мьютексы

Функции для работы с мьютексами

Windows	POSIX
CreateMutex, OpenMutex	pthread_mutex_init
WaitForSingleObject, WaitForMultipleObjects	pthread_mutex_lock pthread_mutex_trylock pthread_mutex_timedlock
ReleaseMutex	pthread_mutex_unlock
CloseHandle	pthread_mutex_destroy

Мьютексы

```
int main()
{

    LPCWSTR             an = L"D:\\VS_Projects\\OS_0001\\Debug\\OS_0031x.exe";
    STARTUPINFO         si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(STARTUPINFO)); si.cb = sizeof(STARTUPINFO);

    HANDLE hm = CreateMutex(NULL, FALSE, L"smwMutex");

    if ( // создать дочернего процесса
        CreateProcess(an, NULL, NULL, NULL, FALSE, NULL, NULL, NULL, &si, &pi)
        )
        std::cout << "-- Process OS_0031x created\n";
    else  std::cout << "-- Process OS_0031x not created\n";

    WaitForSingleObject(hm, INFINITE);
    for (int i = 0; i < 100; i++)
    {
        Sleep(40);    // задержка
        std::cout << "OS_0031: " << i << "\n";
    }
    ReleaseMutex(hm);

    WaitForSingleObject(pi.hProcess, INFINITE);      // ожидание завершения дочернего процесса
    CloseHandle(hm);
    CloseHandle(pi.hProcess);

    return 0;
}
```

```
//OS_0001x.exe
#include <Windows.h>    // для системных вызовов
#include <iostream>

int main()
{
    HANDLE hm = OpenMutex(SYNCHRONIZE, FALSE, L"smwMutex");      //
    if (hm == NULL) std::cout << "OS_0031x: Open Error Mutex \n";
    else std::cout << "OS_0031x: Open Mutex \n";

    WaitForSingleObject(hm, INFINITE);
    for (int i = 0; i < 50; i++)
    {
        Sleep(15);    //
        std::cout << "OS_0031x: " << i << "\n";
    }
    ReleaseMutex(hm);
    CloseHandle(hm);

    return 0;
}
```

Мьютексы vs Критические секции

Сравнение критических секций и мьютексов в Windows

Характеристики	Объект-мьютекс	Объект — критическая секция
Быстродействие	Малое	Высокое
Возможность использования за границами процесса	Да	Нет
Объявление	*HANDLE hmtx;*	*CRITICAL_SECTION cs;*
Инициализация	*hmtx = CreateMutex (NULL, FALSE, NULL);*	*InitializeCriticalSection(&cs);*
Очистка	*CloseHandle(hmtx);*	*DeleteCriticalSection(&cs);*
Бесконечное ожидание	*WaitForSingleObject (hmtx, INFINITE);*	*EnterCriticalSection(&cs);*
Ожидание в течение 0 мс	*WaitForSingleObject (hmtx, 0);*	*TryEnterCriticalSection(&cs);*
Ожидание в течение произвольного периода времени	*WaitForSingleObject (hmtx, dwMilliseconds);*	Невозможно
Освобождение	*ReleaseMutex(hmtx);*	*LeaveCriticalSection(&cs);*
Возможность параллельного ожидания других объектов ядра	Да (с помощью *WaitForMultipleObjects* или аналогичной функции)	Нет

Спинлок и атомарные действия

Также стоит упомянуть, что если вы всё же согласны на присутствие активного ожидания без перехода в состояние ожидания, то можно воспользоваться **спинлоками**

- Спинлок (WinAPI – только на уровне ядра, POSIX)
- Спинлок в критической секции (InitializeCriticalSectionAndSpinCount)

Также можно поработать с атомарными операциями напрямую, если захочется реализовать какой-то примитив на их основе

Атомарные операции (WinAPI, POSIX)

Задача писателя и читателя

Это конечно всё хорошо, но что если разделяемый ресурс не один?

В качестве примера рассмотрим задачу производителя и потребителя (также известную как задача ограниченного буфера)

Два потока используют общий буфер фиксированного размера. Один из них, производитель, помещает информацию в буфер, а другой, потребитель, извлекает ее оттуда

Пусть буфер состоит из N ячеек, каждая из которых может содержать одну запись

Задача писателя и читателя

В общем случае поток-писатель и поток-читатель могут иметь различные скорости и обращаться к буферу с переменной интенсивностью

В один период скорость записи может превышать скорость чтения, в другой – наоборот. Для правильной совместной работы поток-писатель должен приостанавливаться, когда все буферы оказываются занятыми, и активизироваться при освобождении хотя бы одного буфера

Напротив, поток-читатель должен приостанавливаться, когда все буферы пусты, и активизироваться при появлении хотя бы одной записи

Семафоры

В таком случае нам на помощь приходит обобщение блокирующих переменных, так называемые **семафоры Дейкстры** или же просто **семафоры**

Вместо двоичных переменных **Дейкстра** предложил использовать переменные, которые могут принимать произвольные **целые неотрицательные значения**

Такие переменные, используемые для синхронизации вычислительных процессов, получили название **семафоров**

Семафоры

Для работы с семафорами вводятся два примитива, традиционно обозначаемых P и V. Пусть переменная S представляет собой семафор. Тогда действия V(S) и P(S) определяются следующим образом:

- V(S) – Переменная S увеличивается на 1 единым действием. Выборка, наращивание и запоминание не могут быть прерваны. К переменной S нет доступа другим потокам во время выполнения этой операции
- P(S) – Уменьшение S на 1, если это возможно. Если S – 0 и невозможно уменьшить S, оставаясь в области целых неотрицательных значений, то в этом случае поток, вызывающий операцию P, ждет, пока это уменьшение станет возможным. Успешная проверка и уменьшение также является атомарной операцией

Семафоры

Никакие прерывания во время выполнения примитивов V и P недопустимы

В частном случае, когда семафор S может принимать только значения 0 и 1, он превращается в блокирующую переменную, которую по этой причине часто называют **двоичным семафором**

Операция P заключает в себе потенциальную возможность перехода потока, который ее выполняет, в состояние ожидания, в то время как операция V может при некоторых обстоятельствах активизировать другой поток, приостановленный операцией P

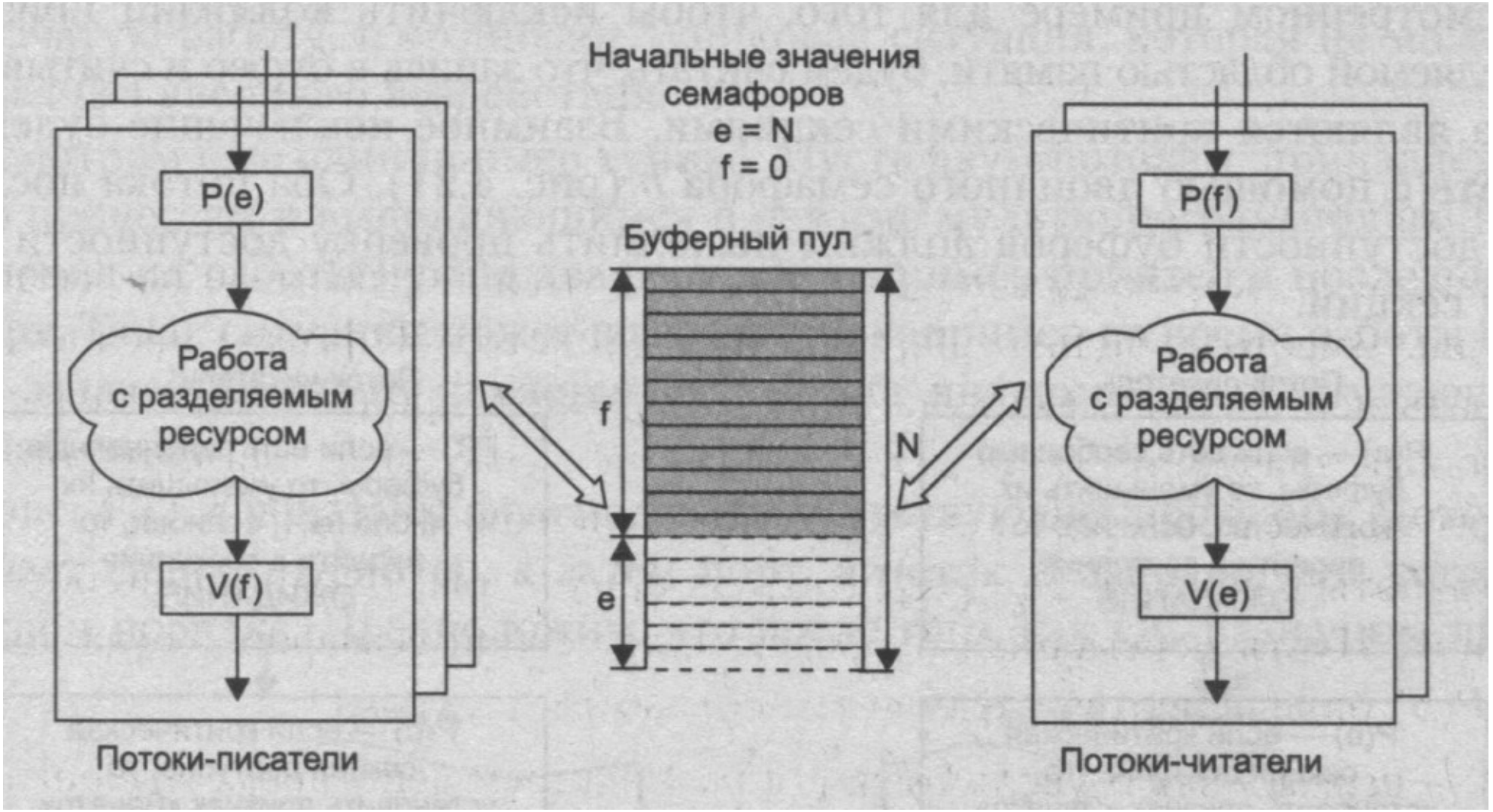
Семафоры

Возвращаясь к задаче, введем два семафора: e – число пустых ячеек буфера и f – число заполненных ячеек буфера, причем в исходном состоянии e = N, а f = 0. Тогда работа потоков с общим буфером может быть описана следующим образом:

Поток-писатель, прежде всего, выполняет операцию P(e), с помощью которой он проверяет, имеются ли в буферном пуле незаполненные буферы. В соответствии с семантикой операции P, если семафор e равен 0 (то есть свободных ячеек в данный момент нет), то поток-писатель переходит в состояние ожидания. Если же значением e является положительное число, то он уменьшает число свободных ячеек, записывает данные в очередную ячейку и после этого наращивает число занятых ячеек операцией V(f)

Поток-читатель действует аналогичным образом с той разницей, что он начинает работу с проверки наличия заполненных ячеек, а после чтения данных наращивает количество свободных ячеек

Семафоры


Начальные значения
семафоров
e = N
f = 0
Буферный пул
P(e)
Работа
с разделяемым
ресурсом
V(f)
f
e
N
P(f)
Работа
с разделяемым
ресурсом
V(e)
Потоки-писатели
Потоки-читатели

Семафоры

В данном случае предпочтительнее использовать семафоры вместо блокирующих переменных. Действительно, критическим ресурсом здесь является буфер, который может быть представлен как набор идентичных ресурсов – отдельных ячеек, а, значит, с буфером могут работать сразу несколько потоков, и именно столько, сколько буферов в нем содержится

Использование двоичной переменной не позволяет организовать доступ к критическому ресурсу более чем одному потоку. Семафор же решает задачу синхронизации более гибко, допуская к разделяемому пулу ресурсов заданное количество потоков

Так, в нашем примере с буфером могут работать максимум N потоков, часть из которых может быть писателями, а часть – читателями

Семафоры

Таким образом, семафоры позволяют эффективно решать задачу синхронизации доступа к ресурсным пулам, таким, например, как набор идентичных в функциональном назначении внешних устройств (модемов, принтеров, портов), или набор областей памяти одинаковой величины, или набор информационных структур

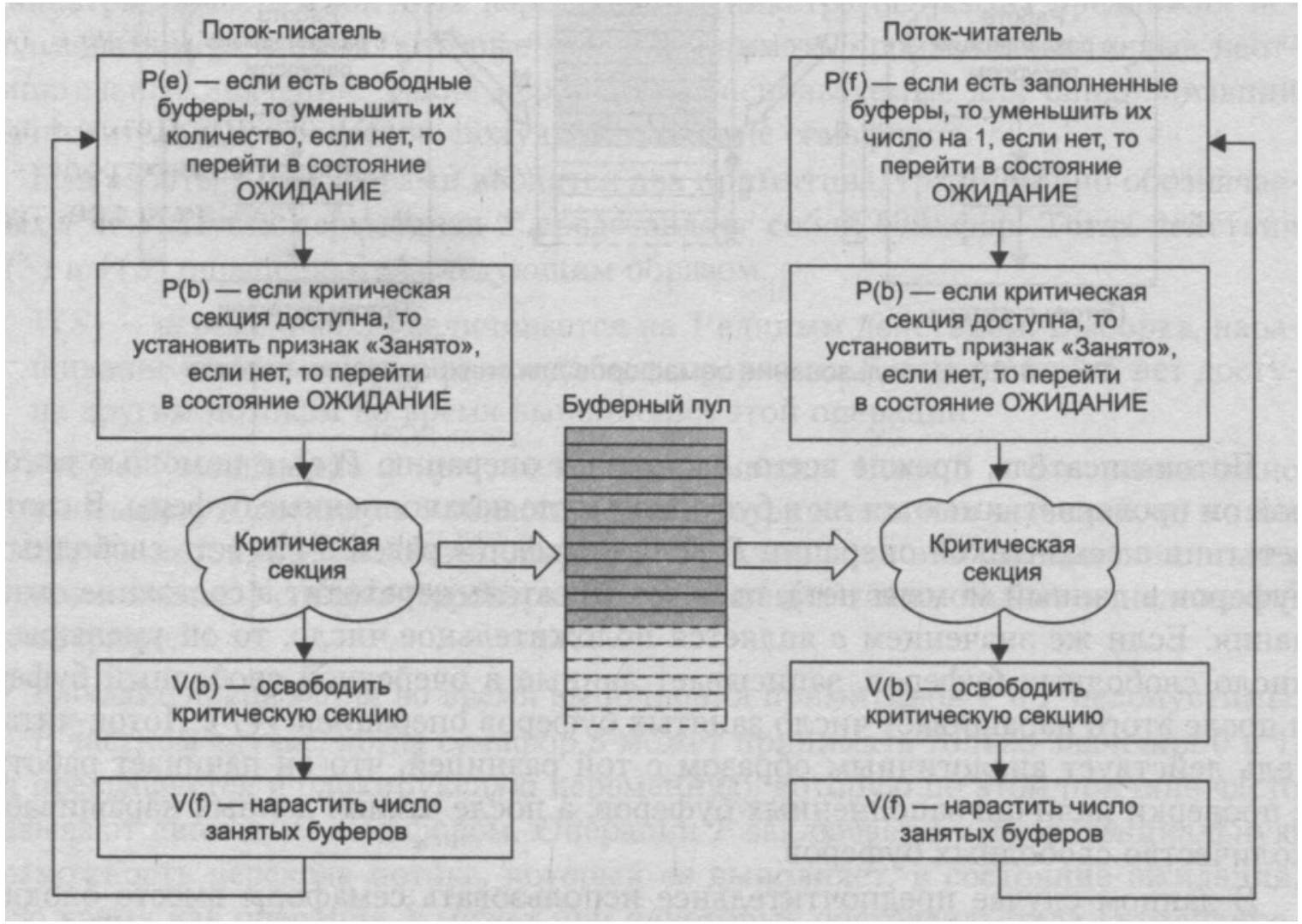
Во всех этих и подобных им случаях с помощью семафоров можно организовать доступ к разделяемым ресурсам **сразу нескольких потоков**

Семафоры

Семафор может использоваться и в качестве блокирующей переменной

В рассмотренном примере для того, чтобы исключить коллизии при работе с разделяемой областью памяти, будем считать, что запись в буфер и считывание из буфера являются критическими секциями. Взаимное исключение будем обеспечивать с помощью двоичного семафора b

Семафоры


Поток-писатель
P(e) — если есть свободные буферы, то уменьшить их количество, если нет, то перейти в состояние ОЖИДАНИЕ
P(b) — если критическая секция доступна, то установить признак «Занято», если нет, то перейти в состояние ОЖИДАНИЕ
Критическая секция
V(b) — освободить критическую секцию
V(f) — нарастить число занятых буферов
Буферный пул
Поток-читатель
P(f) — если есть заполненные буферы, то уменьшить их число на 1, если нет, то перейти в состояние ОЖИДАНИЕ
P(b) — если критическая секция доступна, то установить признак «Занято», если нет, то перейти в состояние ОЖИДАНИЕ
Критическая секция
V(b) — освободить критическую секцию
V(f) — нарастить число занятых буферов

Семафоры

Для управления семафором и WinAPI, и POSIX предоставляют свой набор функций

Windows	POSIX
CreateSemaphore, OpenSemaphore	sem_open (named) / sem_init (unnamed)
WaitForSingleObject, WaitForMultipleObjects	sem_wait sem_getvalue sem_trywait
ReleaseSemaphore	sem_post
CloseHandle	sem_close, sem_unlink (named) / sem_destroy (unnamed)

Семафоры

```
PROCESS_INFORMATION pi1, pi2, pi3;
DWORD pid = GetCurrentProcessId();
// HANDLE hm = CreateMutex(NULL, FALSE, L"smwMutex");
HANDLE hs = CreateSemaphore(NULL, 2, 2, L"smwSem");
LPCWSTR             an = L"D:\\VS_Projects\\OS_0001\\Debug\\OS_0032x.exe";
{
    STARTUPINFO  si; ZeroMemory(&si, sizeof(STARTUPINFO)); si.cb = sizeof(STARTUPINFO);
    if (CreateProcess(an, NULL, NULL, NULL, FALSE, CREATE_NEW_CONSOLE, NULL, NULL, &si,
         std::cout << "-- Process OS_0032x-01 created\n";
    else  std::cout << "-- Process OS_0032x-01 not created\n";
}
{
    STARTUPINFO  si;  ZeroMemory(&si, sizeof(STARTUPINFO)); si.cb = sizeof(STARTUPINFO);
    if (CreateProcess(an, NULL, NULL, NULL, FALSE, CREATE_NEW_CONSOLE, NULL, NULL, &si,
         std::cout << "-- Process OS_0032x-02 created\n";
    else std::cout << "-- Process OS_0032x-02 not created\n";
}
{
    STARTUPINFO  si; ZeroMemory(&si, sizeof(STARTUPINFO)); si.cb = sizeof(STARTUPINFO);
    if (CreateProcess(an, NULL, NULL, NULL, FALSE, CREATE_NEW_CONSOLE, NULL, NULL, &si,
         std::cout << "-- Process OS_0032x-03 created\n";
    else  std::cout << "-- Process OS_0032x-03 not created\n";
}

WaitForSingleObject(hs, INFINITE);
for (int i = 0; i < 1000; i++)
{
    Sleep(40); std::cout << pid << ": OS_0032: " << i << "\n";
}
LONG prevcount = 0;
ReleaseSemaphore(hs, 1, &prevcount);

WaitForSingleObject(pi1.hProcess, INFINITE);      // ожидание завершения дочернего проце
WaitForSingleObject(pi2.hProcess, INFINITE);
WaitForSingleObject(pi2.hProcess, INFINITE);
CloseHandle(hs);
CloseHandle(pi1.hProcess); CloseHandle(pi2.hProcess); CloseHandle(pi3.hProcess);
```

```
int main()
{
    DWORD pid = GetCurrentProcessId();
    HANDLE hs = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, L"smwSem");
    if (hs == NULL) std::cout << "OS_0032x: Open Error Mutex \n";
    else std::cout << "OS_0032x: Open Semaphore \n";

    WaitForSingleObject(hs, INFINITE);
    for (int i = 0; i < 10000; i++)
    {
        Sleep(15);  std::cout <<pid<< ": OS_0032x: " << i << "\n";
    }

    LONG prevcount = 0;
    ReleaseSemaphore(hs, 1, &prevcount);

    CloseHandle(hs);
    std::cout << ": OS_0032x: prevcount  = " << prevcount << "\n";

    system("pause");
    return 0;
}
```

Условные переменные

Ещё одним вариантом решения этой задачи является использование так называемых **условных переменных (conditional variables)**

Для решения задачи самих переменных будет недостаточно, обычно используется в паре с ними некоторый блокирующий механизм который будет оберегать общие данные с которыми ведётся работа несколькими потоками (обычно мьютекс)

Сами же условные переменные используются для информирования о некоторых событиях или о выполнении условий необходимых для продолжения работы программы (в этой задаче например появление данных или освобождение ячейки)

Условные переменные

Мьютексы хороши для разрешения или блокирования доступа к критической области. Условные переменные позволяют потокам блокироваться до выполнения конкретных условий

Если говорить о задаче производителя-потребителя: один поток что-то помещает в буфер, а другой это что-то из него извлекает

Если производитель обнаружил отсутствие в буфере свободных мест, он вынужден блокироваться до тех пор, пока они не появятся. Мьютексы предоставляют возможность производить проверку атомарно, без вмешательства со стороны других потоков, но обнаружив, что буфер заполнен, производитель нуждается в способе блокировки с последующей активизацией

Именно этот способ и предоставляется условными переменными

Условные переменные

Функции для работы с условными переменными

Windows	POSIX
InitializeConditionVariable	pthread_cond_init
SleepConditionVariableCS	pthread_cond_wait
WakeAllConditionVariable	pthread_cond_broadcast
WakeConditionVariable	pthread_cond_signal
	pthread_cond_destroy

В Windows условные переменные используются в критической секцией и не могу использоваться между процессами

RW-lock

Теперь возникает новый вопрос: А что если доступ должен обеспечиваться множеству потоков сразу?

Здесь к нам приходит другая общеизвестная задача – задача читателей и писателей

В ней моделируется доступ к базе данных. Представим, к примеру, систему бронирования авиабилетов, в которой есть множество соревнующихся процессов, желающих обратиться к ней для чтения и записи

Вполне допустимо наличие нескольких потоков, одновременно считывающих информацию из базы данных, но если один поток обновляет базу данных (проводит операцию записи), никакой другой поток не может получить доступ к базе данных даже для чтения информации

Вопрос в том, как создать программу для читателей и писателей?

RW-lock

Самым каноничным решением является использование **RW-локов (Readers–Writer Lock)**

Данный примитив оптимизирован для случая когда у нас очень много операций чтения и изредка происходят операции записи

RW-lock разрешает параллельный доступ для операций, выполняемых только на чтение, тогда как операции записи требуют эксклюзивного доступа. Это означает, что несколько потоков могут считывать данные одновременно, но для записи или изменения данных необходима исключительная блокировка. Когда поток-писатель выполняет запись данных, все остальные писатели и читатели блокируются – вплоть до завершения операции записи

RW-lock

Функции для работы с RW-locks

Windows	POSIX
InitializeSRWLock	pthread_rwlock_init
AcquireSRWLockShared TryAcquireSRWLockShared ReleaseSRWLockShared	pthread_rwlock_rdlock pthread_rwlock_tryrdlock pthread_rwlock_unlock
AcquireSRWLockExclusive TryAcquireSRWLockExclusive ReleaseSRWLockExclusive	pthread_rwlock_wrlock pthread_rwlock_trywrlock pthread_rwlock_unlock
	pthread_rwlock_destroy

RW-lock

Пока есть хотя бы один читатель, никакой писатель не сможет занять RW-lock

Писатели ожидают освобождения всех читателей

В зависимости от реализации читатели могут добавляться даже при имеющейся заявке на блокировку от писателя: приоритет читателя или приоритет писателя

Порядок предоставления блокировки читателям или писателем (все попытки блокировки при её невозможности вносятся в очередь ожидания) не гарантируется и не является честным (не FIFO)

Барьеры

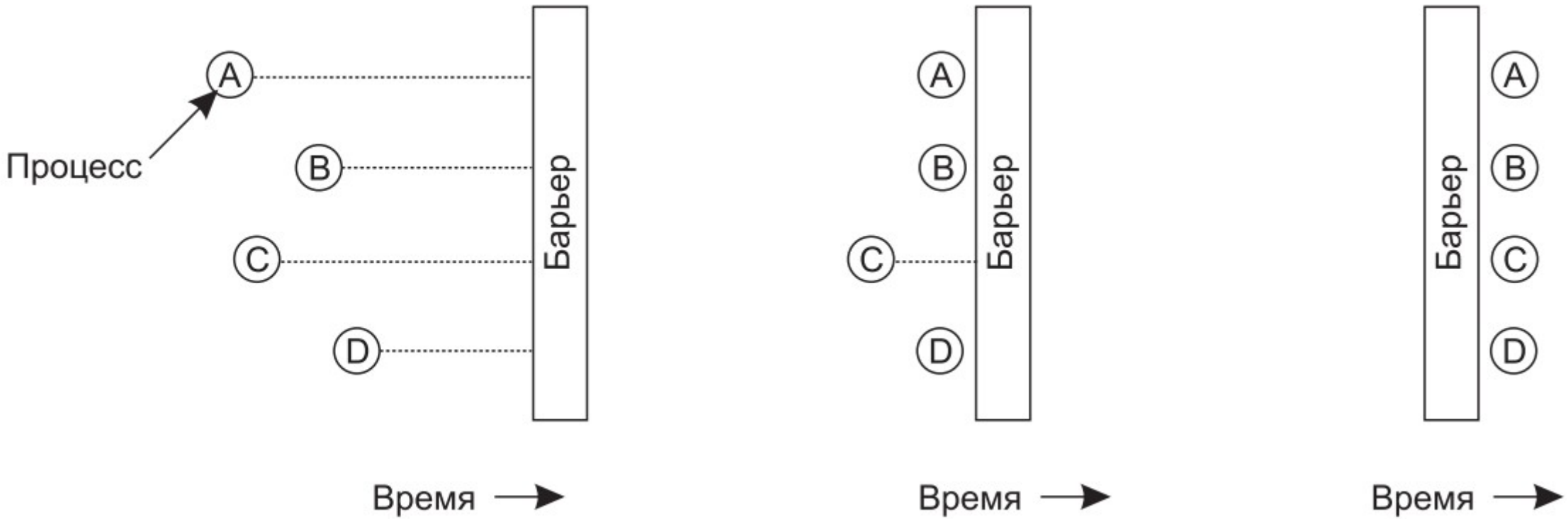
Ещё одним вариантом ситуации когда потребуется особая синхронизация является синхронизация групп потоков

Некоторые приложения разбиты на фазы и следуют правилу, согласно которому ни один из потоков не может перейти к следующей фазе, пока все потоки не будут готовы перейти к следующей фазе

Добиться выполнения этого правила можно с помощью **барьеров**, поставленных в конце каждой фазы. Когда поток достигает барьера, он блокируется до тех пор, пока этого барьера не достигнут все остальные потоки

Это позволяет синхронизировать группы процессов

Барьеры


A
B
C
D
Процесс
Барьер
Время
A
B
C
D
Барьер
Время
Барьер
A
B
C
D
Время

Барьеры

В качестве примера задачи, для решения которой нужны барьеры, рассмотрим обычную в физике или в машиностроении задачу затухания

Обычно это матрица, содержащая некоторые исходные значения. Эти значения могут представлять собой температуру в разных точках листа металла. Замысел может состоять в определении скорости распространения разогрева от пламени, воздействующего на один из его углов

Начиная с текущих значений, к матрице для получения ее следующей версии применяется преобразование, соответствующее, к примеру, законам термодинамики, чтобы посмотреть все температурные показания через время ΔT

Барьеры

Затем процесс повторяется снова и снова, представляя температурные значения в виде функции, зависящей от времени нагревания листа. Со временем алгоритм производит серию матриц, каждая из которых соответствует заданной отметке времени

Теперь представим, что матрица имеет слишком большие размеры (скажем, миллион на миллион) и для ускорения ее вычисления требуется использование параллельных процессов (возможно, на многопроцессорной машине)

Разные процессы работают над разными частями матрицы, вычисляя новые элементы матрицы на основе прежних значений в соответствии с законами физики. Но ни один процесс не может приступить к итерации n + 1, пока не завершится итерация n, то есть пока все процессы не завершат текущую работу

Барьеры

Функции для работы с барьерами

Windows	POSIX
InitializeSynchronizationBarrier	**pthread_barrier_init**
EnterSynchronizationBarrier	**pthread_barrier_wait**
DeleteSynchronizationBarrier	**pthread_barrier_destroy**

События

Кроме уже упомянутых примитивов синхронизации Windows предоставляет еще такой механизм как **События (Events)**

События это такой примитив который позволяет вручную перевести его в сигнальное состояние

Ах да, все объекты синхронизации Windows по сути имеют два состояния сигнальное или несигнальное

По сути когда вы вызываете функцию wait или WaitForSingleObject вы ожидаете перехода какого-либо объекта в сигнальное состояние

События

Существуют два типа объекта события:

Событие сброса вручную. Объект события, состояние которого остается сигнальным до тех пор, пока функция ResetEvent явно не будет сброшена на незначимую. Во время передачи сигнала можно освободить любое количество ожидающих потоков или потоков, которые впоследствии указывают один и тот же объект события в одной из функций ожидания

Событие автоматического сброса. Объект события, состояние которого остается сигнальным до тех пор, пока не будет освобожден один поток ожидания, в этот момент система автоматически устанавливает состояние без знака. Если ожидающих потоков нет, состояние объекта события остается сигнальным

События

Для работы с событиями в WinAPI существуют следующие функции:

- Создание объекта события – **CreateEvent**
- Ожидание сигнального состояния – **WaitForSingleObject**
- Получение существующего объекта по имени – **OpenEvent**
- Установка события в сигнальное состояние – **SetEvent**
- Установка несигнального состояния – **ResetEvent**

События

```
PROCESS_INFORMATION pi1, pi2, pi3;
DWORD pid = GetCurrentProcessId();
HANDLE he = CreateEvent(NULL, FALSE, FALSE, L"smwEvent");
LPCWSTR    an = L"D:\\VS_Projects\\OS_0001\\Debug\\OS_0033x.exe";
{
    STARTUPINFO  si; ZeroMemory(&si, sizeof(STARTUPINFO)); si.cb = sizeof(STARTUPINFO);
    if (CreateProcess(an, NULL, NULL, NULL, FALSE, CREATE_NEW_CONSOLE, NULL, NULL, &si, &pi1
        std::cout << "-- Process OS_0032x-01 created\n";
    else  std::cout << "-- Process OS_0032x-01 not created\n";
}
{
    STARTUPINFO  si;  ZeroMemory(&si, sizeof(STARTUPINFO)); si.cb = sizeof(STARTUPINFO);
    if (CreateProcess(an, NULL, NULL, NULL, FALSE, CREATE_NEW_CONSOLE, NULL, NULL, &si, &pi2
        std::cout << "-- Process OS_0032x-02 created\n";
    else std::cout << "-- Process OS_0032x-02 not created\n";
}
{
    STARTUPINFO  si; ZeroMemory(&si, sizeof(STARTUPINFO)); si.cb = sizeof(STARTUPINFO);
    if (CreateProcess(an, NULL, NULL, NULL, FALSE, CREATE_NEW_CONSOLE, NULL, NULL, &si, &pi3))
        std::cout << "-- Process OS_0032x-03 created\n";
    else  std::cout << "-- Process OS_0032x-03 not created\n";
}
for (int i = 0; i < 3; i++)
{
    Sleep(5000); std::cout << pid << ": OS_0033: " << i << "\n";
    SetEvent(he);

}
WaitForSingleObject(pi1.hProcess, INFINITE);
WaitForSingleObject(pi2.hProcess, INFINITE);
WaitForSingleObject(pi2.hProcess, INFINITE);
CloseHandle(he);
CloseHandle(pi1.hProcess); CloseHandle(pi2.hProcess); CloseHandle(pi3.hProcess);
```

```
int main()
{
    DWORD pid = GetCurrentProcessId();
    HANDLE he = OpenEvent(EVENT_ALL_ACCESS, FALSE, L"smwEvent");
    if (he == NULL) std::cout << "OS_0033x: Open Error Mutex \n";
    else std::cout << "OS_0033x: Open Event \n";

    WaitForSingleObject(he, INFINITE);
    for (int i = 0; i < 10000; i++)
    {
        Sleep(15);  std::cout << pid << ": OS_0032x: " << i << "\n";
    }

    CloseHandle(he);
    system("pause");
    return 0;
}
```

Синхронизация

Синхронизация тесно связана с планированием потоков:

Во-первых, любое обращение потока с системным вызовом вида Wait(X) влечет за собой действия в подсистеме планирования – этот поток снимается с выполнения и помещается в очередь ожидающих потоков, а из очереди готовых потоков выбирается и активизируется новый поток

Во-вторых, при переходе объекта в сигнальное состояние (в результате выполнения некоторого потока – либо системного, либо прикладного) ожидающий этот объект поток (или потоки) переводится в очередь готовых к выполнению потоков

В обоих случаях осуществляется перепланирование потоков, при этом если в ОС предусмотрены изменяемые приоритеты и/или кванты времени, то они пересчитываются по правилам, принятым в этой операционной системе

Синхронизация

Рассмотрим несколько примеров, когда в качестве синхронизирующих объектов используются **файлы**, **потоки** и **процессы**

Пусть программа приложения построена так, что для выполнения запросов, поступающих из сети, основной поток создает вспомогательные серверные потоки. При поступлении от пользователя команды завершения приложения основной поток должен дождаться завершения всех серверных потоков и только после этого завершится сам

Другой пример. Пусть выполнение некоторого приложения требует последовательных работ-этапов. Для каждого этапа имеется свой отдельный процесс. Сигналом для начала работы каждого следующего процесса является завершение предыдущего. Для реализации такой логики работы необходимо в каждом процессе, кроме первого, предусмотреть выполнение системного вызова Wait(X), в котором синхронизирующим объектом является предшествующий поток

Синхронизация

Объект-ƒайл, переход которого в сигнальное состояние соответствует завершению операции ввода-вывода этого ƒайла, используется в тех случаях, когда поток, инициировавший эту операцию, решает дождаться ее завершения прежде, чем продолжить свои вычисления

Однако круг событий, с которыми потоку может потребоваться синхронизировать свое выполнение, отнюдь не исчерпывается завершением потока, процесса или операции ввода-вывода. Поэтому в ОС, как правило, имеются и другие, более универсальные объекты синхронизации, такие как мы рассмотрели ранее в лекции

Операционные системы

Лекция 8

Синхронизация